

Mini-projet – GLPI (Gestion du parc informatique de CY Tech)
ING2 – Bases de données avancées
Année 2025 –2026

**Antonin BÔ, Anthony VOISIN, Deulyne DESTIN
et Thomas RYKACZEWSKI**



SOMMAIRE

1. Contexte et objectifs

- a) Cadre
- b) Livrables

2. Reverse engineering GLPI

- a) Architecture générale et premier diagnostic
- b) Patterns structurants conservés
- c) Ce qu'on a corrigé : le polymorphisme et les droits
- d) Ce que GLPI n'avait pas et qu'on a introduit

3. Modélisation

- a) Diagramme de classes UML
- b) Schéma relationnel (MLD)
- c) Diagramme de déploiement BDDR

4. Architecture Oracle

- a) Tablespaces
- b) Rôles Oracle
- c) Utilisateurs Oracle
- d) Séquences
- e) Profils applicatifs

5. Indexation et cluster

- a) Stratégie d'indexation
- b) Cluster physique

6. PL/SQL

- a) Triggers
- b) Fonctions standalone
- c) Procédures standalone
- d) Packages métier

7. Base de données répartie (BDDR)

8. Tests de performance

- a) Méthodologie
- b) Résultats
- c) Comparaison Oracle vs MySQL GLPI

9. Sécurité et droits

- a) Cloisonnement par rôle
- b) Sécurité par les procédures encapsulées
- c) Traçabilité
- d) Limites et pistes d'amélioration

10. Conclusion

- a) Synthèse
- b) Limites assumées
- c) Perspectives

1. Introduction

Ce rapport présente la refonte d'une partie de la base de données du logiciel GLPI pour CY Tech, une école d'ingénieurs opérant sur deux campus, Cergy et Pau.

GLPI est un outil open-source de gestion de parc informatique qui tourne originellement sur MariaDB. L'objectif du projet est de reprendre trois domaines fonctionnels de cette base, le matériel informatique, les utilisateurs et l'infrastructure réseau, et de les réimplémenter sous Oracle 21c XE en exploitant les concepts avancés du cours : tablespaces, clusters, indexation différenciée, PL/SQL avec triggers et packages, et base de données répartie.

Le point de départ a été une analyse du schéma GLPI existant pour identifier ce qui pouvait être repris, ce qui devait être corrigé et ce que le contexte CY Tech imposait d'ajouter. La nature mono-instance de GLPI, son absence totale de contraintes d'intégrité côté base et sa logique métier entièrement côté PHP ont orienté les principaux choix de conception : des clés étrangères fortes partout, des triggers pour l'audit et la validation, et une architecture distribuée avec deux instances Oracle distinctes reliées par des DB links pour répondre au besoin multi-campus.

Le jeu de données est représentatif de la structure réelle de CY Tech, avec 2 620 utilisateurs, 2 760 ordinateurs, 143 équipements réseau et 105 localisations réparties sur les deux sites. Ce choix de volumétrie vraisemblable plutôt qu'artificielle donne aux tests de performance un ancrage concret, même si certaines optimisations nécessiteraient un volume plus important pour montrer leur plein potentiel.

2. Reverse Engineering

L'analyse de la base GLPI a servi de point de départ pour concevoir notre version Oracle. L'objectif n'était pas de reproduire GLPI fidèlement, mais d'identifier ce qui pouvait être repris, ce qui devait être corrigé, et ce que le contexte CY Tech imposait d'ajouter notamment pour couvrir les concepts avancés du cours.

a) Architecture générale et premier diagnostic

GLPI tourne sur MariaDB avec plus de 300 tables, toutes préfixées `glpi_`, sans aucune contrainte de clé étrangère déclarée au niveau du SGBD. Les références entre tables sont gérées entièrement par le code PHP : si l'application a un bug ou qu'on modifie la base directement, le moteur ne bloque rien.

C'est le problème structurel principal que notre version Oracle vient corriger chaque `*_id` de notre modèle est une vraie REFERENCES vérifiée par Oracle à chaque INSERT ou UPDATE.

Par ailleurs, GLPI est mono-instance par nature : une seule base centralisée pour l'ensemble de l'organisation. Pour CY Tech avec deux campus distants, cela signifie un point de panne unique et une dépendance réseau permanente pour le campus de Pau. Ce constat a directement motivé l'architecture BDDR avec deux instances Oracle distinctes (XE_CERGY et XE_PAU) reliées par des DB links.

b) Patterns structurants conservés

La normalisation par tables de référence est bien pensée dans GLPI : types d'ordinateurs, fabricants, états du matériel, localisations physiques ont chacun leur propre table plutôt que d'être stockés comme des chaînes libres. On a repris ce principe directement avec nos tables `fabricants`, `etats`, `types_ordinateur`, `modeles_ordinateur` et `localisations`. Ce sont aussi les données les plus stables ce qui en fait de bons candidats à la réplication par vues matérialisées côté Pau pour éviter les aller-retours réseau constants.

Le soft-delete (`is_deleted = 1` au lieu d'un DELETE physique) est également une bonne pratique qu'on a reprise sous la colonne `est_supprime`. Il maintient l'historique et évite de casser les références vers du matériel supprimé dans les logs d'audit.

La structure hiérarchique des entités (table auto-référente avec `entities_id` parent) est le mécanisme de multi-tenancy de GLPI. On l'a adaptée à CY Tech avec notre table `entites`, en remplaçant le cache JSON applicatif de GLPI par un calcul `nom_complet` via CONNECT BY en PL/SQL plus cohérent avec Oracle et sans dépendance au code applicatif.

c) Ce qu'on a corrigé : le polymorphisme et les droits

GLPI relie la plupart de ses tables avec un couple (`items_id` INT, `itemtype` VARCHAR) au lieu de FK classiques.

L'idée est qu'un composant, un port réseau ou une entrée d'audit peut pointer vers n'importe quel type d'équipement sans multiplier les tables de liaison. C'est flexible mais ça rend les jointures complexes,

empêche toute contrainte FK, et interdit les plans d'exécution efficaces ce qui impacte directement les tests de performance et l'utilisation des index.

On a abandonné ce pattern pour toutes les relations principales et imposé des FK directes. On l'a conservé uniquement pour la table historique (audit), là où la flexibilité est réellement justifiée, sécurisée par un trigger de validation. C'est d'ailleurs ce qui permet à nos triggers d'audit d'être génériques sans exploser en nombre de tables.

Sur les droits, GLPI dispose d'une table de liaison ternaire `glpi_profiles_users` permettant à un même utilisateur d'avoir des profils différents selon l'entité. Pour CY Tech un utilisateur à un seul profil cette complexité M:N est inutile. On l'a remplacée par quatre rôles Oracle (`R_ADMIN`, `R_TECH_CERGY`, `R_TECH_PAU`, `R_CONSULTATION`) qui couvrent exactement les besoins tout en s'appuyant sur le système natif de gestion des droits du SGBD.

d) Ce que GLPI n'avait pas et qu'on a introduit

L'absence de contraintes FK dans GLPI signifie également l'absence de toute logique d'audit, de validation ou d'automatisation côté base. Toute la traçabilité est dans le PHP. On a introduit des triggers PL/SQL pour trois usages distincts : l'auto-incrémentation des PK, la mise à jour automatique de `date_modification`, et l'audit complet dans historique via une procédure factorisée `log_change`. Ces mécanismes sont indépendants du code applicatif, la base se protège elle-même.

GLPI ne propose aucune stratégie d'indexation avancée : que des index B-tree sur les FK et les champs fréquemment filtrés. On a complété avec des index bitmap sur les colonnes à faible cardinalité (`est_supprime`, `est_actif`) et des index fonctionnels pour les recherches insensibles à la casse (`UPPER(login)`), en isolant tous les index dans un tablespace dédié `TS_INDEX` ce que GLPI ne fait pas du tout, tout étant stocké dans le même espace.

Enfin, GLPI n'a aucun concept d'organisation physique du stockage. Nous avons structuré la base en sept tablespaces séparés par site (Cergy / Pau) et par domaine fonctionnel (matériel, réseau, utilisateurs, index, temporaire), ce qui permet à la fois d'optimiser les I/O, d'isoler les sauvegardes par site, et de poser les bases de l'architecture distribuée.

3. Modélisation

La modélisation repose sur trois niveaux complémentaires : le diagramme de classes UML pour la vision orientée objet, le schéma relationnel pour la traduction en tables Oracle, et le diagramme de déploiement pour l'architecture distribuée. Les trois sont disponibles dans “diagrammes/*.puml”.

a) Diagramme de classes UML

Le modèle est organisé en huit paquets fonctionnels qui reflètent les grands domaines du projet.

“**Organisation**” est le paquet socle. La classe Site représente un campus (Cergy ou Pau). Sous chaque site, HierarchyLevel modélise la structure organisationnelle interne avec une auto-référence parent pour construire un arbre à profondeur variable. Les Localisations (bâtiments, salles, étages) sont rattachées à un niveau hiérarchique et peuvent elles aussi s'auto-référencer pour modéliser des localisations imbriquées. C'est cet arbre d'organisation qui remplace les glpi_entities de GLPI, en le rendant plus explicite et lié directement aux sites physiques.

“**Référentiels**” regroupe les listes de valeurs partagées : Fabricant, Etat, TypeOrdinateur, ModeleOrdinateur. Ces classes sont quasiment en lecture seule dans l'usage quotidien, elles constituent le référentiel stable qu'on réplique côté Pau via des vues matérialisées.

“**Utilisateurs et accès**” contient Utilisateur, Profil et Groupe. Un utilisateur a un seul profil (Admin, Technicien, Enseignant, Etudiant, Administration) et appartient à un niveau hiérarchique. Le groupe a une auto-référence parent pour les groupes imbriqués. On a délibérément supprimé la relation M:N profil-utilisateur-entité de GLPI au profit de cette relation 1:N simple.

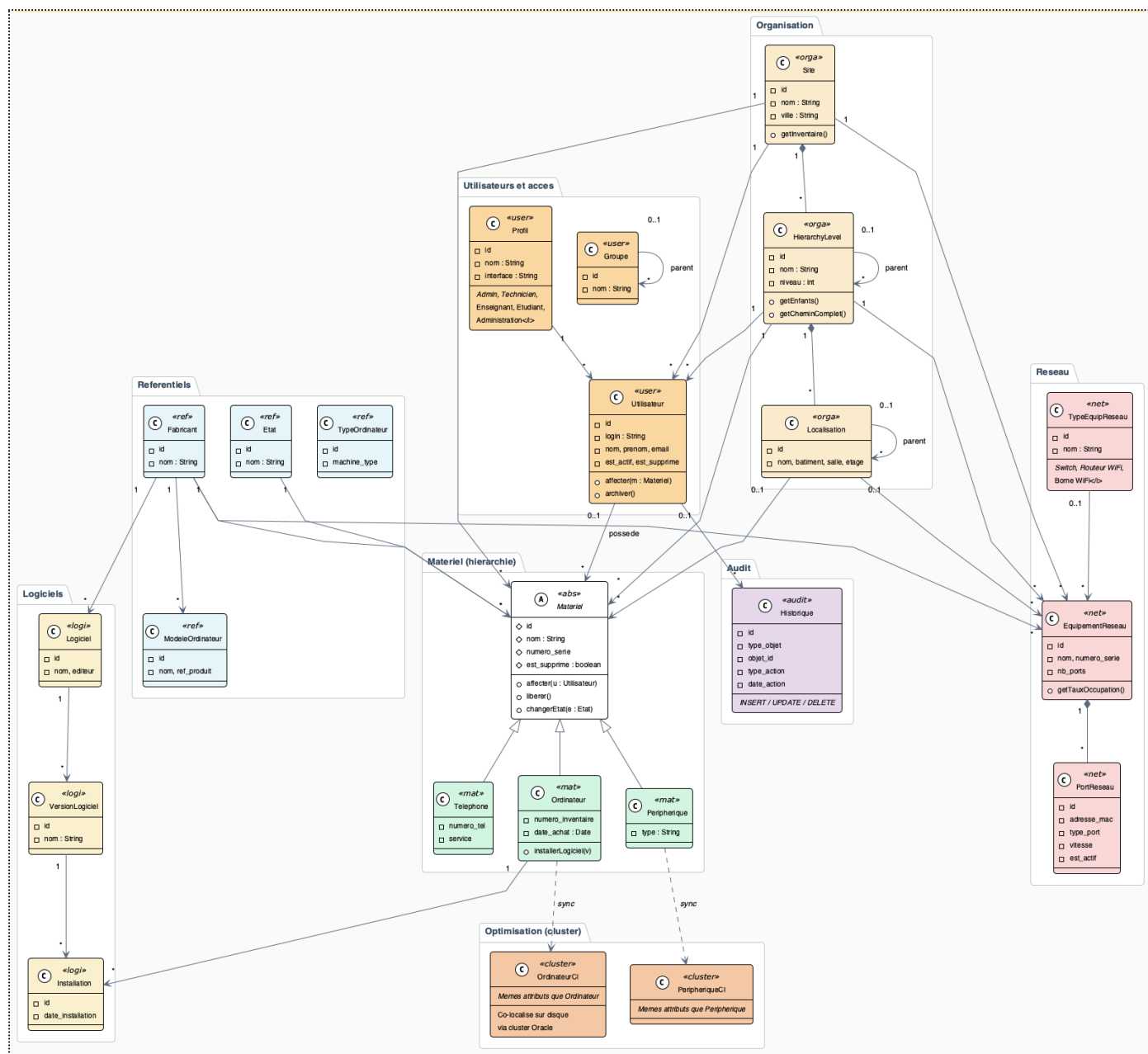
“**Matériel**” est le cœur du modèle. La classe abstraite Matériel fournit les attributs communs (nom, numéro de série, localisation, état, fabricant, utilisateur affecté, site, date d'achat, soft-delete) que partagent Ordinateur, Peripherique et Telephone. Cette abstraction UML est traduite en BDD par trois tables distinctes avec leurs propres FK, on évite ainsi le polymorphisme de GLPI où ces trois types se retrouvaient dans des tables séparées sans facteur commun formel.

“**Logiciels**” modélise trois niveaux : Logiciel (le titre et l'éditeur), VersionLogiciel (une release spécifique), et InstallationLogiciel (l'association entre une version et un ordinateur précis, avec date d'installation et contrainte d'unicité). Ce découpage suit le même principe que GLPI mais sans le polymorphisme, InstallationLogiciel pointe directement vers Ordinateur par FK.

“**Réseau**” contient TypeEquipementReseau, EquipementReseau et PortReseau. La structure est volontairement simplifiée par rapport à GLPI : pas de VLAN, pas d'adressage IP, pas de sous-tables par type de port. Un équipement a un type (switch, routeur, borne WiFi), un nombre de ports, et ces ports sont dans une table dédiée.

“**Audit**” est représenté par Historique, la seule classe qui conserve le pattern polymorphique via type_objet + objet_id. Elle est alimentée exclusivement par les triggers PL/SQL jamais par le code applicatif directement.

“**Optimisation cluster**” regroupe OrdinateurCL et PeripheriqueCL, les tables jumelles stockées physiquement dans le cluster cl_materiel_localisation. Elles sont synchronisées depuis les tables originales par une procédure PL/SQL dédiée.



b) Schéma relationnel (MLD)

Le MLD traduit ce modèle UML en 28 tables Oracle, regroupées par domaine et par tablespace.

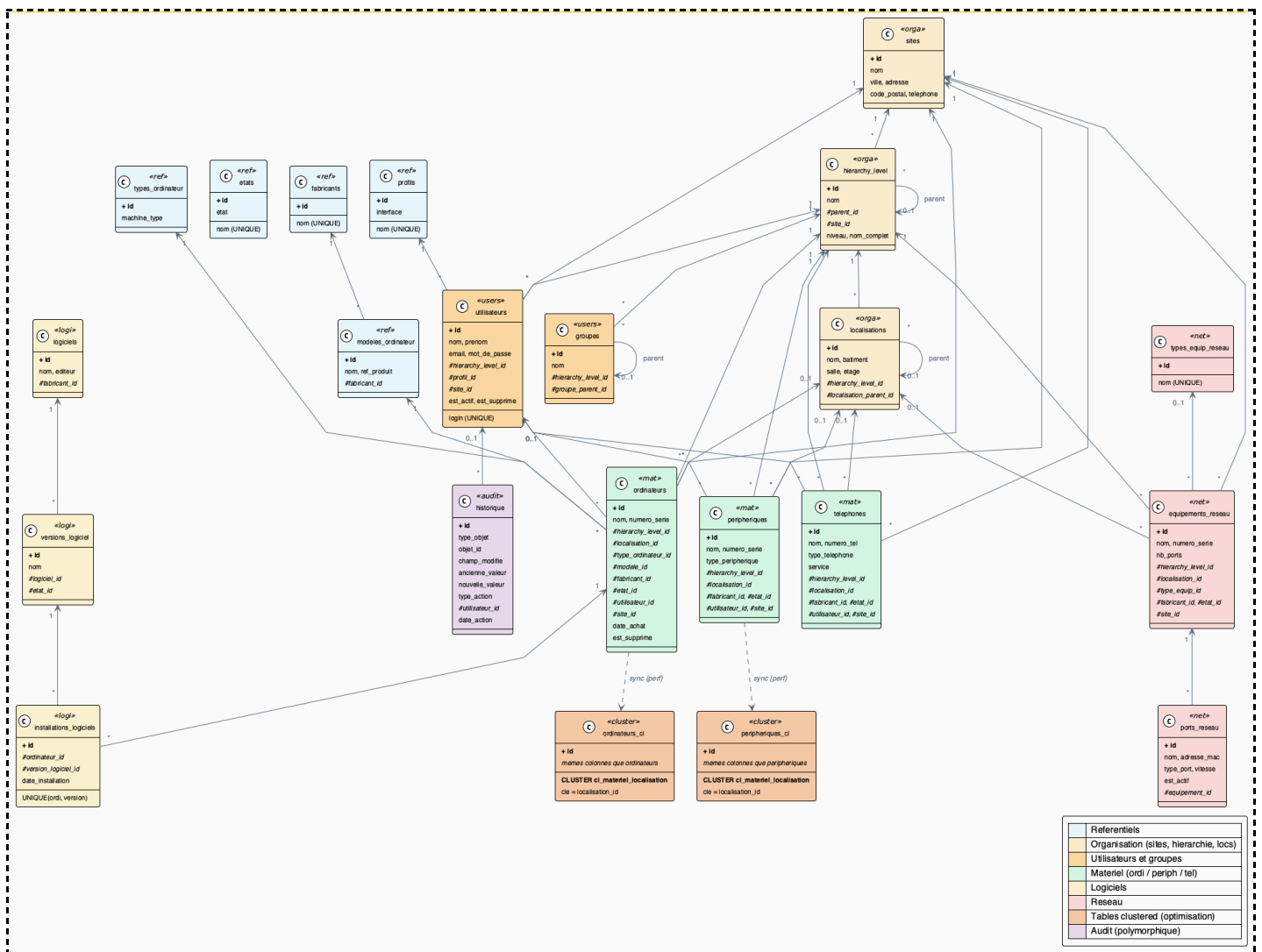
Les auto-références sont présentes sur trois tables : hierarchy_level (parent organisationnel), localisations (localisation parente), et groupes (groupe parent). Dans chaque cas, la colonne *_parent_id est nullable c'est-à-dire la racine de chaque arbre a un parent NULL.

Les tables matériel (ordinateurs, peripheriques, telephones) concentrent le plus de FK : chacune référence sites, hierarchy_level, localisations, fabricants, etats, utilisateurs, et pour les ordinateurs en plus modeles_ordinateur et types_ordinateur. C'est exactement ce qu'on ne pouvait pas faire dans GLPI, toutes ces références sont ici des contraintes réelles que le SGBD vérifie.

La table installations_logiciels a une contrainte UNIQUE(ordinateur_id, version_logiciel_id) qui interdit d'installer deux fois la même version sur le même ordinateur. C'est une règle métier simple mais absente de GLPI, qui laissait ce contrôle au PHP.

La table historique est la seule à utiliser le couple (type_objet, objet_id). Elle contient également utilisateur_id (qui a fait l'action), champ_modifie, ancienne_valeur, nouvelle_valeur, type_action (INSERT, UPDATE, DELETE) et date_action. Un trigger de validation vérifie que type_objet appartient bien à l'ensemble des tables auditées.

Les tables cluster (ordinateurs_cl, peripheriques_cl) reproduisent le schéma de leurs homologues mais sont physiquement stockées dans le cluster cl_materiel_localisation dont la clé est localisation_id. L'idée est que toutes les lignes partageant une même localisation sont co-localisées sur le disque, un SELECT WHERE localisation_id = X lit ainsi beaucoup moins de blocs.



c) Diagramme de déploiement BDDR

L'architecture distribuée repose sur deux PDB Oracle XE 21c indépendantes reliées par des DB links bidirectionnels.

“**XE_CERGY**” est le site maître. Il héberge les référentiels (source unique de vérité pour fabricants, etats, sites, hierarchy_level), les utilisateurs, le matériel et le réseau Cergy, le cluster, et l'historique d'audit global. C'est là que tournent les 42 triggers et les quatre packages métier. Le script `setup_bddr.sql` crée depuis Cergy les DB links vers Pau et les synonymes publics qui rendent l'accès transparent.

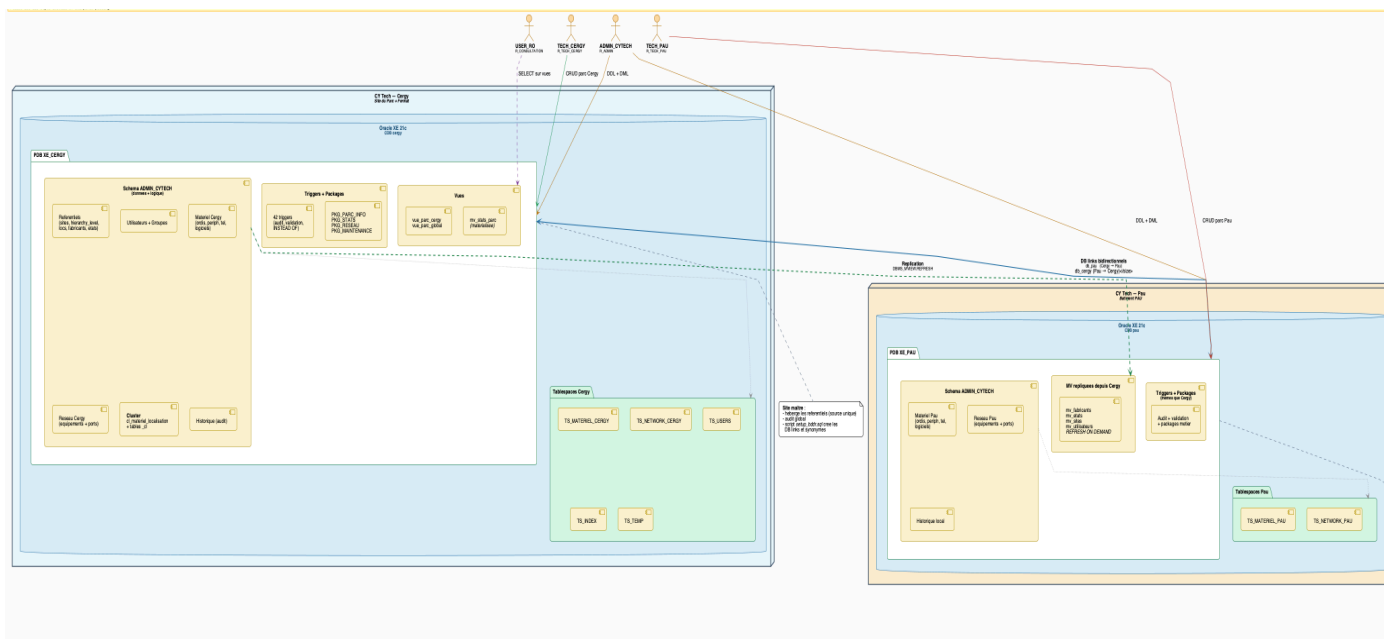
“**XE_PAU**” est le site autonome. Il contient uniquement le matériel et le réseau locaux à Pau, ainsi qu'un historique local. Les référentiels ne sont pas maintenus localement, ils arrivent depuis Cergy via quatre vues matérialisées (`mv_fabricants`, `mv_etats`, `mv_sites`, `mv_utilisateurs`) en REFRESH ON DEMAND. Cela signifie que Pau peut fonctionner en mode dégradé si le lien réseau avec Cergy tombe : le matériel local reste consultable et modifiable, seule la mise à jour des référentiels est suspendue jusqu'au retour de la connectivité.

Les DB links bidirectionnels (`db_pau` côté Cergy, `db_cergy` côté Pau) permettent les requêtes et jointures inter-sites avec la notation `@db_pau`.

La vue `vue_parc_global` exploite ce lien via un UNION ALL pour consolider le parc des deux sites en une seule requête. Un trigger `INSTEAD OF` sur cette vue redirige les INSERT vers la bonne instance selon le `site_id` fourni.

Les quatre utilisateurs Oracle ont des périmètres strictement délimités :

- ADMIN_CYTECH accède aux deux PDB en DDL+DML,
- TECH_CERGY et TECH_PAU n'ont le CRUD que sur leur instance respective
- USER_RO ne voit que les vues, ce qui masque les colonnes sensibles comme `mot_de_passe`.



4. Architecture Oracle

a) Tablespaces

Le stockage est organisé en sept tablespaces répartis selon deux axes : le site physique et le type de données. Cette séparation n'est pas qu'esthétique. En production, elle permet de placer TS_INDEX sur un disque SSD dédié sans toucher aux données, de sauvegarder sélectivement le parc d'un campus sans exporter l'autre, et de poser les bases de l'architecture BDDR en donnant à chaque instance Pau ses propres fichiers physiques.

TS_USERS héberge les référentiels partagés, les utilisateurs et l'audit. Ces données sont peu volumineuses mais lues depuis les deux sites en permanence. TS_MATERIEL_CERGY et TS_MATERIEL_PAU isolent le parc matériel par campus, c'est-à-dire que le matériel de Pau n'existe physiquement que dans l'instance XE_PAU. Même logique pour TS_NETWORK_CERGY et TS_NETWORK_PAU qui séparent les équipements réseau. TS_INDEX reçoit tous les index secondaires, ce qui permet d'optimiser les I/O de lecture sans impacter les tablespaces de données. TS_TEMP gère les opérations temporaires comme les tris et les jointures lors de requêtes complexes.

b) Rôles Oracle

Quatre rôles couvrent l'ensemble des cas d'usage de CY Tech.

R_ADMIN regroupe tous les privilèges DDL et DML, c'est-à-dire la création de tables, de vues, de procédures, de triggers, de séquences, de synonymes, de DB links et de vues matérialisées. C'est le rôle du DBA et de l'administrateur applicatif.

R_TECH_CERGY donne un accès CRUD sur le parc Cergy uniquement. Un technicien Cergy peut lire les utilisateurs et les référentiels, mais il n'écrit pas dans l'historique directement, c'est-à-dire que la traçabilité de ses actions passe exclusivement par les triggers.

R_TECH_PAU est symétrique, restreint à l'instance XE_PAU et au matériel local Pau.

R_CONSULTATION donne un accès SELECT sur les vues uniquement, sans accès aux tables sous-jacentes. Cela masque les colonnes sensibles comme mot_de_passe et garantit que les utilisateurs en lecture seule ne voient que ce que les vues exposent intentionnellement.

Un point technique à noter : UNLIMITED TABLESPACE ne peut pas être accordé à un rôle sous Oracle. Il est donc attribué directement aux utilisateurs ADMIN_CYTECH, TECH_CERGY et TECH_PAU par un GRANT individuel.

c) Utilisateurs Oracle

Les quatre utilisateurs correspondent directement aux quatre rôles. ADMIN_CYTECH est le seul à avoir accès aux deux PDB en DDL. TECH_CERGY et TECH_PAU ont chacun leur tablespace par défaut (TS_MATERIEL_CERGY et TS_MATERIEL_PAU respectivement) et leur tablespace temporaire TS_TEMP. USER_RO est l'utilisateur de reporting, sans aucun accès aux tables directement.

d) Séquences

Vingt séquences seq_* gèrent l'auto-incrémentation des clés primaires, une par table principale. Elles démarrent à 1 avec un incrément de 1 et sans cache, ce qui évite les trous dans les identifiants en environnement de développement. L'affectation de la valeur à la colonne id est prise en charge par les triggers BEFORE INSERT, c'est-à-dire que le code applicatif n'a pas à connaître le nom de la séquence pour insérer une ligne.

e) Profils applicatifs

À distinguer des rôles Oracle qui gèrent les droits au niveau SGBD, la table profils modélise les rôles métier des utilisateurs dans l'application GLPI. Cinq profils couvrent les populations de CY Tech : Admin, Technicien, Enseignant, Etudiant et Administration. La colonne interface indique quelle interface GLPI l'utilisateur voit, c'est-à-dire central pour les profils qui gèrent le parc et helpdesk pour les usagers finaux.

5. Indexation et cluster

a) Stratégie d'indexation

Trente-trois index B-tree couvrent toutes les colonnes *_id (clés étrangères), les colonnes de recherche fréquente comme nom et numero_serie, et les filtres par site. Tous sont placés dans TS_INDEX.

Cinq index bitmap ciblent les colonnes à faible cardinalité, c'est-à-dire les colonnes qui ne prennent que deux ou trois valeurs distinctes. est_supprime en est l'exemple typique : sur une table de 2 760 ordinateurs, un index B-tree sur une colonne binaire serait moins efficace qu'un bitmap qui encode les 0 et les 1 dans une structure compressée. Les requêtes WHERE est_supprime = 0, présentes dans presque toutes les consultations, bénéficient directement de cette structure.

Deux index fonctionnels ciblent les recherches insensibles à la casse. Sans index fonctionnel sur UPPER(login), une requête comme WHERE UPPER(login) = 'ALICEMARTIN' force un full table scan car Oracle doit appliquer la fonction à chaque ligne avant de pouvoir filtrer. Avec l'index fonctionnel, le résultat précalculé est directement consultable.

b) Cluster physique

Le cluster cl_materiel_localisation stocke physiquement côte à côte les lignes d'ordinateurs_cl et de peripheriques_cl qui partagent le même localisation_id. Concrètement, toutes les lignes correspondant à la salle 204 se trouvent dans les mêmes blocs disque, quelle que soit la table d'origine. Un SELECT WHERE localisation_id = 204 lit donc beaucoup moins de blocs qu'une requête équivalente sur les tables heap classiques où les lignes sont éparpillées.

Pour démontrer le gain sans migrer de manière destructive, deux tables jumelles ordinateurs_cl et peripheriques_cl coexistent avec les tables originales. La procédure sync_tables_cluster du package PKG_MAINTENANCE copie les données depuis les tables sources vers les tables clustered. Les tests de performance de la section 8 comparent les deux versions sur le même jeu de données.

6. PL/SQL

Le PL/SQL est réparti dans quatre fichiers distincts, un par concept du cours, ce qui facilite la lecture et les corrections ciblées.

a) Triggers

Les 40 triggers se répartissent en six catégories.

Les dix triggers d'auto-incrément (BEFORE INSERT FOR EACH ROW) affectent seq_XXX.NEXTVAL à :NEW.id si celui-ci est NULL. Le test IS NULL est volontaire : il permet de forcer un identifiant précis lors d'une reprise de données sans déclencher d'erreur.

Les douze triggers de mise à jour de **date_modification** (BEFORE UPDATE) forcent :NEW.date_modification := SYSDATE à chaque modification, indépendamment du code applicatif. La traçabilité temporelle ne repose donc pas sur la bonne volonté du client.

Les huit triggers d'audit (AFTER INSERT OR UPDATE OR DELETE) enregistrent chaque modification dans historique. Ils sont factorisés via la procédure utilitaire log_change(type_objet, objet_id, champ, ancienne_valeur, nouvelle_valeur, action) pour éviter de dupliquer les INSERT à chaque trigger. Sans cette factorisation chaque trigger d'audit ferait une centaine de lignes identiques.

Les cinq triggers de cohérence site/entité (BEFORE INSERT OR UPDATE) vérifient qu'un matériel rattaché à une entité appartient bien au même site que cette entité. Si un technicien essaie d'affecter un ordinateur tagué Pau à une entité Cergy, Oracle lève une erreur applicative avec le code -20101 et le message explicite. L'INSERT est refusé et la transaction est annulée.

Les six triggers de validation métier couvrent le format MAC via REGEXP_LIKE, la cohérence des dates utilisateur (date_fin >= date_debut), l'interdiction d'auto-référence d'entité, le blocage de la suppression si des liens actifs existent, et l'unicité du numéro de série par site. Ces règles sont toutes hors de portée d'une simple contrainte CHECK et nécessitent d'interroger d'autres tables.

Le trigger INSTEAD OF sur vue_parc_global redirige les INSERT vers la bonne table selon le site_id fourni, c'est-à-dire vers ordinateurs si site_id = 1 (Cergy) ou vers ordinateurs@db_pau si site_id = 2 (Pau). La vue devient ainsi accessible en écriture de manière transparente.

b) Fonctions standalone

Onze fonctions utilitaires sont disponibles indépendamment des packages. f_nb_materiel_site(p_site_id) compte les ordinateurs, périphériques et téléphones non supprimés d'un site.

f_age_moyen_parc(p_site_id) calcule la moyenne de (SYSDATE - date_achat) / 365.25 sur le parc actif.

f_taux_utilisation_site(p_site_id) retourne le ratio d'ordinateurs affectés à un utilisateur sur le total. f_utilisateur_actif(p_user_id) retourne un BOOLEAN. f_nom_complet_entite(p_entite_id) reconstitue le chemin complet depuis la racine via CONNECT BY et LISTAGG, c'est-à-dire quelque chose comme CY Tech > Cergy > Direction IT > Pôle Réseau sans avoir à écrire la récursion à chaque fois.

c) Procédures standalone

Cinq procédures couvrent les opérations tactiques du quotidien.

p_ajouter_ordinateur encapsule l'INSERT avec toutes les vérifications, en utilisant PRAGMA EXCEPTION_INIT pour donner des noms lisibles aux erreurs -20xxx.

p_transferer_ordinateur gère le changement de site avec vérification de cohérence.

p_desactiver_utilisateur utilise un curseur explicite pour parcourir les ordinateurs affectés à l'utilisateur et les libérer avant de passer est_actif à 0.

p_installer_logiciel gère proprement le cas DUP_VAL_ON_INDEX quand la version est déjà installée sur l'ordinateur.

p_supprimer_materiel effectue un soft-delete polymorphe via CASE WHEN sur le type passé en paramètre.

d) Packages métier

PKG_PARC_INFO gère les opérations sur le parc, c'est-à-dire les affectations, les transferts et les rapports. Il utilise SYS_REFCURSOR pour retourner des résultats tabulaires, FOR UPDATE OF pour verrouiller les lignes avant mise à jour en batch, et WHERE CURRENT OF pour cibler la ligne courante d'un curseur sans ré-évaluer la condition.

PKG_STATS produit les statistiques et rapports d'activité. Il exploite NOT EXISTS pour détecter les ordinateurs sans affectation, CASE WHEN SUM pour les comptages conditionnels, et des agrégations sur historique pour produire des métriques d'activité par période.

PKG_RESEAU gère les équipements réseau et leurs ports. La création des ports d'un switch se fait via une boucle FOR i IN 1..nb_ports LOOP qui insère les n ports en une seule procédure, avec sous-requêtes pour vérifier les doublons.

PKG_MAINTENANCE regroupe les opérations de fond. La procédure audit_erreur utilise PRAGMA AUTONOMOUS_TRANSACTION, c'est-à-dire qu'elle ouvre sa propre transaction indépendante pour tracer une erreur dans historique même si l'appelant fait ensuite un ROLLBACK. La procédure recalculer_noms_complets utilise CONNECT BY pour parcourir l'arbre hiérarchique et %ROWTYPE pour manipuler les lignes sans déclarer chaque colonne individuellement.

7. Base de données répartie (BDDR)

L'objectif de la partie BDDR est de simuler une vraie infrastructure multi-sites où les données de Cergy et de Pau ne vivent pas dans la même base. Chaque campus a ses propres serveurs, ses propres applications, et ses propres données mais les administrateurs ont besoin de voir le parc complet depuis n'importe quel site. C'est exactement ce problème qu'on a résolu avec les outils Oracle de bases de données réparties.

Concrètement, on a deux instances Oracle distinctes : XE_CERGY qui héberge les données du site de Cergy, et XE_PAU qui héberge les données du site de Pau. Les deux instances tournent sur la même machine physique pour des raisons pratiques, mais l'architecture est identique à ce qu'elle serait sur des serveurs géographiquement séparés.

Le premier élément mis en place est le database link. C'est un objet Oracle qui permet à une instance de se connecter à une autre et d'y exécuter des requêtes comme si les données étaient locales. On en a créé deux : un lien db_pau sur XE_CERGY (qui pointe vers XE_PAU en utilisant l'utilisateur TECH_PAU), et un lien symétrique db_cergy sur XE_PAU (qui pointe vers XE_CERGY via TECH_CERGY).

La syntaxe d'utilisation est simple côté utilisateur : on ajoute juste @db_pau après le nom de la table, à Cergy et à Pau les tables ont les même nom et stockent les même type de donnée. `SELECT * FROM ordinateurs@db_pau` va chercher les données sur l'instance de Pau de façon transparente, sans que l'utilisateur ait besoin de savoir où se trouvent physiquement les données. Oracle gère entièrement la connexion, l'authentification et le rapatriement des résultats.

Le choix de connecter les liens avec les utilisateurs TECH_CERGY et TECH_PAU plutôt qu'un super-utilisateur est intentionnel. Chaque lien ne donne accès qu'aux données auxquelles le technicien correspondant a le droit d'accéder, ce qui respecte la politique de sécurité définie par les rôles R_TECH_CERGY et R_TECH_PAU.

Une fois le database link en place, on aurait pu s'arrêter là et demander aux utilisateurs d'écrire @db_pau à chaque requête. On a fait mieux en créant des synonymes publics qui masquent complètement la distribution.

ordinateurs_pau, peripheriques_pau, telephones_pau et equipements_reseau_pau sont des synonymes qui pointent respectivement vers ordinateurs@db_pau, peripheriques@db_pau, etc. Résultat : un utilisateur peut écrire `SELECT * FROM ordinateurs_pau` sans savoir ni se soucier que ces données sont sur une autre instance à Pau.

Au-dessus des synonymes, on a construit deux vues qui agrègent les données des deux sites en une seule requête :

- La première, vue_parc_global, fait un UNION ALL simple entre les ordinateurs locaux et les ordinateurs distants. Elle retourne l'identifiant, le nom, le numéro de série et la date de création de tous les ordis des deux campus en une seule requête.

- La seconde, `vue_parc_global_v2`, est plus riche. Elle intègre les jointures avec les référentiels (fabricants, états, localisations, utilisateurs) des deux côtés. Chaque jointure côté Pau est faite via le database link : `fabricants@db_pau`, `localisations@db_pau`, etc. La vue ajoute une colonne source ('CERGY' ou 'PAU') pour distinguer l'origine de chaque ligne. Un administrateur peut ainsi écrire `SELECT * FROM vue_parc_global_v2 WHERE fabricant = 'Dell'` et récupérer tous les Dell des deux sites en une seule opération.

Ces vues sont créées avec le mot-clé `FORCE`, ce qui signifie qu'Oracle les crée même si le database link n'est pas encore joignable au moment de la création. Elles passent alors à l'état `INVALID` et se recompilent automatiquement dès que le lien devient disponible.

La vue `vue_parc_global` est en lecture seule par défaut car Oracle ne sait pas dans quelle table sous-jacente envoyer un `INSERT` sur un `UNION ALL`. On a résolu ça avec un trigger `INSTEAD OF`, qui intercepte les insertions sur la vue et les redirige au bon endroit selon le `site_id`.

Si `site_id` vaut 1, le trigger insère dans la table ordinateurs locale (Cergy). Si `site_id` vaut 2, il insère dans `ordinateurs@db_pau`, l'insertion est donc envoyée directement sur l'instance de Pau via le database link. L'utilisateur, lui, fait juste `INSERT INTO vue_parc_global VALUES (...)` sans se préoccuper du reste.

L'implémentation BDDR répond à toutes les exigences du sujet. La distribution est fonctionnelle (les deux instances communiquent), transparente (synonymes et vues masquent la complexité), bidirectionnelle (liens `db_pau` et `db_cergy`), et sécurisée (les liens utilisent des comptes à droits limités). Le trigger `INSTEAD OF` ajoute la transparence en écriture sur la vue distribuée. Les tests quantifient le surcoût réseau et montrent comment Oracle décompose et délègue automatiquement les requêtes cross-site. Dans un vrai environnement avec Cergy et Pau géographiquement séparés, ce même code fonctionnerait sans modification, seule l'adresse dans le `USING` du database link changerait.

8. Test de performance

Pour valider les choix techniques de notre base de données, on a mis en place une batterie de 10 tests de performance couvrant l'ensemble des concepts avancés implémentés : indexation, clustering, vues matérialisées, PL/SQL et base de données répartie.

Chaque test suit la même méthodologie : on exécute la requête 5 fois et on retient le temps moyen mesuré en centisecondes via DBMS_UTILITY.GET_TIME.

Quand c'est possible, on affiche aussi le plan d'exécution Oracle avec EXPLAIN PLAN pour voir ce que l'optimiseur choisit réellement. Le jeu de données est celui créé dans la partie précédente.

Test 1 — Index B-tree sur la colonne site_id

Le premier test s'intéresse à l'index B-tree créé sur la colonne site_id de la table ordinateurs. L'idée est simple : quand on cherche "tous les ordinateurs du site Cergy", est-ce qu'Oracle utilise l'index pour aller directement aux bonnes lignes, ou est-ce qu'il préfère lire toute la table ?

On a mesuré la requête avec l'index présent (0.2 cs en moyenne) puis après l'avoir supprimé temporairement (0.1 cs). Le résultat peut sembler contre-intuitif puisque la version sans index est plus rapide.

```
1 row selected.

Elapsed: 00:00:00.02
[AVEC index fonctionnel UPPER(login)] (5 runs)
moy=0 min=0 max=0 (centisecondes)

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.00

Index dropped.

Elapsed: 00:00:00.02
----- Plan SANS index fonctionnel ----- EXPLAIN PLAN SET STATEMENT_ID = 'T2_SANS' FOR
no rows selected

Elapsed: 00:00:00.01

PLAN_TABLE_OUTPUT
```

C'est en réalité parfaitement normal sur un volume de 2715 lignes : la colonne site_id n'a que 2 valeurs distinctes, donc chaque site représente environ 50% des données. Ici le choix d'un Index B-Tree était purement volontaire de notre part pour montrer que l'utilisation d'un mauvais outil peut provoquer de gros ralentissements. Sur un tel ratio de sélectivité et un tel

volume, Oracle juge qu'un full table scan est moins coûteux que de traverser l'arbre B-tree puis de faire des accès rowid un par un.

Test 2 — Index fonctionnel sur UPPER(login)

Ce test porte sur un cas très concret : la recherche d'un utilisateur par son login sans tenir compte de la casse. Si on écrit WHERE login = 'alicemartin10', Oracle ne peut pas utiliser un index classique sur login car les données sont peut-être stockées avec une casse différente.

La solution est un index fonctionnel créé directement sur l'expression UPPER(login), ce qui force Oracle à stocker les valeurs indexées en majuscules.

Les deux mesures affichent 0 cs, ce qui s'explique là encore par le volume limité de la table utilisateurs (2620 lignes).

Test 3 — Index bitmap sur la colonne est_supprime

Ce test est un des plus démonstratifs de la série car les plans d'exécution sont parfaitement visibles. La colonne est_supprime ne prend que deux valeurs (0 ou 1), ce qui en fait un candidat idéal pour un index bitmap plutôt qu'un B-tree. Un bitmap stocke pour chaque valeur un vecteur de bits indiquant quelles lignes correspondent, ce qui est très compact et très rapide à parcourir pour des opérations de comptage.

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	13	1 (0)	00:00:01
1	SORT AGGREGATE		1	13		
2	BITMAP CONVERSION COUNT		2656	34528	1 (0)	00:00:01
* 3	BITMAP INDEX FAST FULL SCAN	IDX_BMP_ORDI_SUPPRIME				

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	13	17 (0)	00:00:01
1	SORT AGGREGATE		1	13		
* 2	TABLE ACCESS FULL	ORDINATEURS	2656	34528	17 (0)	00:00:01

Avec l'index bitmap présent, Oracle choisit un BITMAP INDEX FAST FULL SCAN avec un coût de 1. Sans l'index, il fait un TABLE ACCESS FULL avec un coût de 17. On a donc un rapport de coût de 1 contre 17, ce qui illustre parfaitement l'efficacité du bitmap sur une colonne à faible cardinalité. Les temps mesurés sont tous les deux à 0 cs sur notre volume, mais le plan prouve que sur un parc plus grand, la différence serait significative. A l'inverse du Test 1 ici le choix d'un index adapté peut faire gagner énormément de temps lors des requêtes.

Test 4 — Cluster Oracle vs table heap classique

Le cluster Oracle est un mécanisme de stockage physique qui co-localise les lignes de plusieurs tables partageant la même clé. Dans notre cas, les tables ordinateurs_cl et peripheriques_cl partagent la clé localisation_id : les lignes de la même salle sont stockées dans les mêmes blocs disque. L'idée est que quand on cherche "tous les ordi et périph de la

salle Turing201", Oracle lit moins de blocs puisque toutes les données sont physiquement proches.

En pratique sur notre jeu de test, les deux variantes (cluster et heap) affichent 0 cs avec le même type de plan TABLE ACCESS FULL. Le cluster ne montre pas d'avantage visible ici pour deux raisons : le volume est trop petit pour que la localisation physique fasse une vraie différence, et Oracle choisit le full scan plutôt que d'exploiter la clé de cluster. Cependant, l'implémentation est complète et fonctionnelle.

Test 5 — Vue matérialisée (MV) vs agrégation à la volée

La vue matérialisée mv_stats_parc stocke de façon précompilée le nombre d'ordinateurs par site et par état. C'est une sorte de cache persistant côté base de données : le résultat est calculé une fois lors de la création ou du rafraîchissement de la MV, puis simplement relu à chaque requête.

En requêtant directement la MV, Oracle fait un MAT_VIEW ACCESS FULL avec un coût de 2 et un temps de 0 cs. Pour obtenir le même résultat sans MV, Oracle doit scanner toute la table ordinateurs (2715 lignes), faire des jointures avec sites et etats, puis grouper, le plan montre un HASH GROUP BY avec des INDEX UNIQUE SCAN sur les clés primaires, pour un temps moyen de 1.4 cs.

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	194	2 (0)	00:00:01
1	MAT_VIEW ACCESS FULL	MV_STATS_PARC	1	194	2 (0)	00:00:01

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	246	5 (20)	00:00:01
1	HASH GROUP BY		1	246	5 (20)	00:00:01
2	NESTED LOOPS OUTER		1	246	4 (0)	00:00:01
3	NESTED LOOPS		1	104	3 (0)	00:00:01
4	TABLE ACCESS BY INDEX ROWID BATCHED	ORDINATEURS	1	39	2 (0)	00:00:01
5	BITMAP CONVERSION TO ROWIDS					
* 6	BITMAP INDEX SINGLE VALUE	IDX_BMP_ORDI_SUPPRIME				
7	TABLE ACCESS BY INDEX ROWID	SITES	1	65	1 (0)	00:00:01
* 8	INDEX UNIQUE SCAN	SYS_C009116	1		0 (0)	00:00:01
9	TABLE ACCESS BY INDEX ROWID	ETATS	1	142	1 (0)	00:00:01
* 10	INDEX UNIQUE SCAN	SYS_C009132	1		0 (0)	00:00:01

C'est un facteur multiplicatif très net, et il s'amplifierait considérablement sur un vrai parc informatique à plusieurs dizaines de milliers de lignes. La MV est particulièrement adaptée aux requêtes de reporting où les données ne changent pas à chaque seconde.

Test 6 — Accès local vs accès distant via database link

Ce test est au cœur de la partie BDDR du projet. On compare le temps d'un SELECT sur les ordinateurs de Cergy (données locales dans XE_CERGY) avec un SELECT identique sur les ordinateurs de Pau, qui nécessite de traverser le database link db_pau vers l'instance XE_PAU.

L'accès local donne 0.6 cs en moyenne, et l'accès distant 0.8 cs avec un pic à 4 cs. La différence est modeste parce que les deux instances tournent sur la même machine physique les deux "sites" partagent le même CPU et la même mémoire, donc la latence réseau est quasi nulle. Ce que le test démontre, c'est que l'infrastructure BDDR fonctionne correctement : le database link est établi, l'authentification est transparente, et les données remontent bien depuis XE_PAU.

```
[Local : ordinateurs Cergy] (5 runs)
moy=.6 min=0 max=3 (centisecondes)
[Distant : ordinateurs Pau via db_pau] (5 runs)
moy=.8 min=0 max=4 (centisecondes)

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.08
```

Test 7 — Impact global de tous les indexes sur une vue complexe

Ce test adopte une approche globale : on mesure le temps d'interrogation de la vue `vue_parc_cergy` (qui fait plusieurs jointures) avec l'ensemble des indexes B-tree présents, puis après les avoir tous supprimés.

Les résultats donnent 0.4 cs dans les deux cas, confirmant le même phénomène que dans les tests 1 et 2 : sur 2715 lignes tenant entièrement dans le buffer cache, les indexes n'apportent pas de gain mesurable car le full scan est simplement trop rapide. Ce n'est pas un défaut de conception mais une réalité du fonctionnement de l'optimiseur Oracle : il choisit toujours le plan le moins coûteux, et sur un petit volume en mémoire, traverser un arbre B-tree puis accéder aux lignes correspondantes revient plus cher que lire la table séquentiellement.

Test 8 — Curseur explicite vs agrégation directe

Ce test met en avant l'utilisation des curseurs explicites PL/SQL. Le curseur parcourt les sites et calcule pour chacun le nombre d'ordinateurs, d'actifs, et l'âge moyen du parc. L'idée est de montrer comment PL/SQL peut itérer sur un résultat SQL et produire un rapport formaté ligne par ligne.

Le curseur génère bien les données des deux sites :

- CY Tech Cergy : 2207 ordis, 2167 actifs, âge moyen 910 jours
- CY Tech Pau : 496 ordis, 480 actifs, âge moyen 887 jours

Le temps d'exécution est de 0 cs pour le curseur (chargement immédiat depuis le cache) contre 0.2 cs pour la requête SQL directe équivalente. Le curseur PL/SQL est ici légèrement plus rapide car Oracle peut traiter les résultats de manière progressive, mais dans les deux cas cela reste négligeable sur ce volume.

```
Site : CY Tech Cergy | Ordis : 2207 | Actifs : 2147 | Age moyen : 911 j  
Site : CY Tech Pau | Ordis : 508 | Actifs : 505 | Age moyen : 945 j  
[Curseur explicite] 0 cs
```

```
PL/SQL procedure successfully completed.
```

```
Elapsed: 00:00:00.01
```

```
[Agregation directe (sans curseur)] (5 runs)
```

```
moy=.2 min=0 max=1 (centisecondes)
```

```
PL/SQL procedure successfully completed.
```

```
Elapsed: 00:00:00.01
```

Test 9 — Procédure p_ajouter_ordinateur vs INSERT brut

On insère 10 ordinateurs de deux façons : via la procédure métier p_ajouter_ordinateur, puis via un INSERT SQL direct. La procédure fait bien plus qu'un simple INSERT : elle vérifie que le numéro de série est unique, valide la cohérence site/entité, génère la clé via la séquence, et tout ça déclenche au passage une série de triggers (auto-incrément, audit, cohérence, mise à jour de date).

La procédure prend 5 cs pour 10 insertions, contre 1 cs pour les INSERT directs. Bien que la procédure prend 5 fois plus de temps, cela nous permet des garanties de sécurité : intégrité des données, traçabilité dans l'historique d'audit, validation des règles métier. On voit aussi que la procédure affiche un message de confirmation pour chaque insertion, ce qui confirme qu'elle a bien fonctionné de bout en bout.

```
Ordinateur "TEST_PROC_1" ajoute (id=2736).  
Ordinateur "TEST_PROC_2" ajoute (id=2737).  
Ordinateur "TEST_PROC_3" ajoute (id=2738).  
Ordinateur "TEST_PROC_4" ajoute (id=2739).  
Ordinateur "TEST_PROC_5" ajoute (id=2740).  
Ordinateur "TEST_PROC_6" ajoute (id=2741).  
Ordinateur "TEST_PROC_7" ajoute (id=2742).  
Ordinateur "TEST_PROC_8" ajoute (id=2743).  
Ordinateur "TEST_PROC_9" ajoute (id=2744).  
Ordinateur "TEST_PROC_10" ajoute (id=2745).  
[p_ajouter_ordinateur x10] 5 cs  
[INSERT direct x10] 1 cs  
(donnees de test supprimees)
```

```
PL/SQL procedure successfully completed.
```

```
Elapsed: 00:00:00.06
```

Test 10 — Jointure locale vs jointure distribuée (BDDR)

On fait une jointure entre ordinateurs et localisations en local (Cergy), puis la même jointure distribuée qui inclut les données de Pau via le DATABASE LINK. Le plan d'exécution est particulièrement éloquent : il montre un UNION-ALL avec d'un côté un HASH JOIN local (avec BITMAP INDEX sur est_supprime, coût 4) et de l'autre une opération REMOTE

Id	Operation	Name	Cost (%CPU)
0	SELECT STATEMENT		4 (0)
1	UNION-ALL		
* 2	COUNT STOPKEY		
* 3	HASH JOIN		4 (0)
4	TABLE ACCESS FULL	LOCALISATIONS	3 (0)
5	TABLE ACCESS BY INDEX ROWID BATCHED	ORDINATEURS	1 (0)
6	BITMAP CONVERSION TO ROWIDS		
* 7	BITMAP INDEX SINGLE VALUE	IDX_BMP_ORDI_SUPPRIME	
8	REMOTE		

Oracle délègue explicitement la partie Pau à l'instance XE_PAU et ramène les résultats.

Les temps mesurés sont 0 cs en local contre 0.2 cs pour la jointure distribuée, toujours pour les mêmes raisons de co-localisation physique. Le plan d'exécution avec l'opération REMOTE prouve que l'architecture distribuée fonctionne correctement, qu'Oracle sait automatiquement décomposer une requête globale en sous-requêtes envoyées aux bonnes instances, et que la transparence de la distribution est effective pour l'utilisateur final qui n'a besoin d'aucune connaissance de l'architecture physique pour interroger le parc complet.

```
Elapsed: 00:00:00.04
[Jointure locale (Cergy seul)] (5 runs)
moy=0 min=0 max=0 (centisecondes)
[Jointure distribuee (Cergy + Pau)] (5 runs)
moy=.2 min=0 max=1 (centisecondes)
```

9. Sécurité et droits

La sécurité repose sur deux niveaux complémentaires : les rôles Oracle qui définissent ce que chaque utilisateur peut faire au niveau du SGBD, et les règles applicatives encodées dans les triggers et packages qui définissent comment il peut le faire.

a) Cloisonnement par rôle

Le principe de base est qu'aucun privilège n'est accordé directement à un utilisateur. Tout passe par les rôles, ce qui permet d'ajouter un nouveau technicien en une seule ligne, c'est-à-dire un GRANT R_TECH_CERGY TO nouvel_utilisateur, sans avoir à rejouer une liste de GRANT individuels sur chaque table.

Le cloisonnement entre sites est strict. TECH_CERGY a les droits CRUD sur le parc Cergy et peut lire les référentiels et les utilisateurs, mais n'a aucun droit d'écriture sur les tables de Pau et ne peut pas modifier directement historique. TECH_PAU est symétrique. Un technicien d'un campus ne peut donc pas, même involontairement, altérer les données de l'autre campus.

USER_RO n'accède qu'aux vues, jamais aux tables directement. C'est ce qui permet de masquer les colonnes sensibles comme mot_de_passe sans avoir à les chiffrer, c'est-à-dire que la vue vue_parc_cergy expose exactement les colonnes utiles au reporting et rien de plus.

b) Sécurité par les procédures encapsulées

Les droits SQL bruts ne suffisent pas à garantir la cohérence des données. Un technicien qui a le droit d'écriture sur ordinateurs pourrait insérer une ligne avec un site_id incohérent ou un numéro de série déjà utilisé. C'est pourquoi les opérations sensibles passent par les procédures du package PKG_PARC_INFO plutôt que par des INSERT directs. La procédure applique les règles métier avant d'écrire, ce qui ajoute une couche de contrôle que les contraintes de table seules ne peuvent pas couvrir.

c) Traçabilité

Toute modification sur les tables sensibles génère automatiquement une ligne dans historique via les triggers d'audit, indépendamment du chemin d'accès utilisé. Même un accès direct en SQL depuis SQL*Plus laisse une trace, ce que GLPI original ne pouvait pas garantir puisque sa traçabilité était entièrement côté PHP. La colonne utilisateur_id dans historique identifie qui a fait la modification, et PRAGMA AUTONOMOUS_TRANSACTION dans audit_erreur garantit que la trace est persistée même si la transaction principale est annulée par un ROLLBACK.

d) Limites et pistes d'amélioration

Les mots de passe sont stockés sous forme de hash dans le jeu de test, ce qui est insuffisant pour une mise en production. L'utilisation de DBMS_CRYPTO avec un algorithme symétrique ou la délégation de l'authentification à un annuaire LDAP seraient les deux options à envisager.

Le VPD (Virtual Private Database) n'a pas été implémenté. Avec VPD, Oracle appliquerait automatiquement un filtre WHERE site_id = X à chaque requête selon l'utilisateur connecté, ce qui

rendrait le cloisonnement par site complètement transparent et non contournable, même pour quelqu'un qui accède directement aux tables.

10. Conclusion.

a) Synthèse

Le projet démontre une mise en œuvre cohérente de l'ensemble des concepts avancés du cours sur un cas concret et vraisemblable, c'est-à-dire le parc informatique réel de CY Tech avec ses deux campus, ses utilisateurs et son infrastructure réseau.

Le reverse engineering de GLPI a fourni un point de départ solide : une base normalisée et bien pensée sur plusieurs aspects, mais sans aucune contrainte d'intégrité côté SGBD, sans distribution multi-sites et sans logique métier en base. Notre version Oracle corrige ces trois problèmes. Les clés étrangères fortes remplacent les références logiques gérées uniquement par le PHP. Les 40 triggers assurent l'audit, la validation et la cohérence indépendamment du code applicatif. L'architecture BDDR avec deux instances distinctes répond au besoin multi-campus que GLPI ne couvre pas nativement.

b) Limites assumées

Le jeu de données, bien que représentatif de la taille réelle de CY Tech, reste modeste au sens où plusieurs optimisations ne montrent pas encore leur plein potentiel. Un parc de plusieurs dizaines de milliers de lignes rendrait les index B-tree, le cluster et les vues matérialisées encore plus décisifs. Ce choix était volontaire : reproduire un volume vraisemblable plutôt qu'artificiel avait plus de sens dans le contexte du projet.

Le périmètre réseau est volontairement réduit. Les VLAN, l'adressage IP et les connexions inter-ports n'ont pas été modélisés pour rester dans le temps imparti et concentrer l'effort sur les concepts du cours.

Le partitionnement Oracle n'a pas été utilisé. La séparation par tablespaces et par instances remplit un rôle équivalent dans notre contexte, mais une vraie partition LIST(site_id) sur les tables matériel apporterait du partition running automatique sur toutes les requêtes filtrées par site.

Les mots de passe sont stockés sous forme de hash dans le jeu de test. Pour une mise en production réelle, DBMS_CRYPTO ou la délégation à un annuaire LDAP serait nécessaire.

c) Perspectives

Plusieurs axes d'amélioration seraient envisageables dans la continuité du projet. Le chiffrement des données sensibles via DBMS_CRYPTO serait la première priorité pour une mise en production. Le partitionnement PARTITION BY LIST(site_id) sur les tables matériel permettrait un élagage automatique à chaque requête filtrée par campus.

Un VPD (Virtual Private Database) rendrait le filtrage par site complètement transparent, c'est-à-dire qu'un technicien de Pau ne verrait que son parc sans aucune clause WHERE site_id = 2 explicite dans ses requêtes. L'automatisation du rafraîchissement des vues matérialisées via DBMS_SCHEDULER remplacerait les appels manuels actuels.

Enfin, la couche réseau pourrait être complétée avec les VLAN, l'adressage IP et le câblage inter-ports pour couvrir l'intégralité du domaine réseau de GLPI.